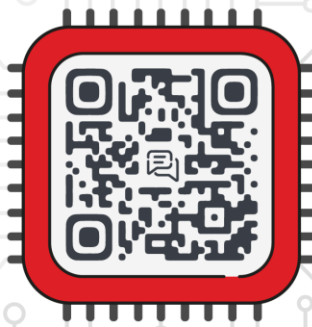




نظري

5

19

2850 sp

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Software Testing Techniques & Coverage

د. سعيد أبو طراب



RB Informatics; 13/05/2026

محتوى مجاني غير مخصص للبيع التجاري

هندسة برمجيات ٢

Testing

:Strategies of Testing

1. White Box Testing

عن طريق:

1. Control Flow Testing

2. Data Flow Testing

2. Black Box Testing

اختبار الصندوق الأبيض White Box Testing

أولاً: CFT



```
1. input (x, y);
2. z=0;
3. while (x+y>0) {
4.     if (x>0)
5.         x=x-1;
6.     if (y>0)
7.         y=y-1;
8.     z=z+x+y;
9. }
10. output (z);
```

مثال 1:

طريقة الحل:

1. رسم ال CFG

2. إيجاد ال paths حسب ال Criteria

3. إيجاد ال test cases

ما يهمنا هو الأسطر التي تمثل التعليمات (Statements)
لدينا هنا إزالة للأسطر الأول والثاني والثالث، فهي مجرد تسلسل من التعليمات (Sequence of Statements)

بمجرد الوصول إلى السطر الثالث، نلاحظ نقطتين أساسيتين:

- النقطة الأولى: لدينا شرط (Condition) يجب أن يتحقق حتى يدخل التنفيذ إلى هذا الجزء .
- النقطة الثانية: لدينا حلقة تكرار (Loop) ، ويجب أن نعرف حدود هذه الحلقة .

نلاحظ أن السطر (3) يمثل بداية الحلقة، وتنتهي الحلقة عند السطر (8)
أما الشرط فهو:

$$0 < y + x$$

إذا تحقق الشرط وإذا لم يتحقق، فإن أي شرط يُعد دائماً تفرعاً (Branch)

بالتالي:

- إذا تحقق الشرط في السطر (3) ينتقل التنفيذ إلى السطر (4)
- وإذا لم يتحقق ينتقل مباشرة إلى السطر (9)

وبالتالي:

- نقطة الإدخال (Input) هي السطر (1)
- ونقطة الإخراج (Output) هي السطر (9)

قبل الدخول في تفاصيل الحلقة، نلاحظ أيضاً أنه يوجد داخل الحلقة شرطان إضافيان.
في السطر الرابع:

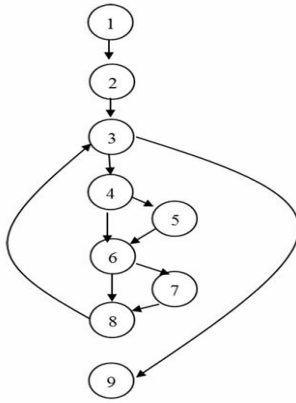
- إذا تحقق الشرط ينتقل التنفيذ إلى السطر (5) وبعد الانتهاء من السطر (5) ينتقل التنفيذ إلى السطر (6)
- وإذا لم يتحقق ينتقل إلى السطر (6) مباشرة

والأمر نفسه في السطر السادس:

- إذا تحقق الشرط ينتقل إلى السطر (7) وبعد الانتهاء من السطر (7) ينتقل التنفيذ إلى السطر (8)
- وإذا لم يتحقق ينتقل إلى السطر (8) مباشرة

بعد الانتهاء من السطر (8)، يعود التنفيذ إلى شرط الحلقة من جديد:

- إذا تحقق الشرط يدخل إلى الحلقة مرة أخرى .
- وإذا لم يتحقق يخرج من الحلقة .



وهكذا نحصل على مخطط تدفق التحكم (Control Flow Graph - CFG)

2. إيجاد ال paths حسب ال Criteria

الآن سنبدأ باستخراج اختبار المسارات (Path Testing) ، وسنعمد على ثلاثة معايير للتغطية:

- تغطية التعليمات (Statement Coverage)
- تغطية التفرعات (Branch Coverage)
- تغطية المسارات (Path Coverage)

1. تغطية التعليمات (Statement Coverage):

أساس تغطية التعليمات (Statement Coverage) هو زيارة جميع العقد. المطلوب هو إيجاد أقل عدد من مسارات الاختبار (Testing Paths) التي تغطي جميع العقد وفق نسبة التغطية المطلوبة. المسار:

1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 3 → 9

يمكنه وحده تغطية جميع التعليمات.

يمكن كتابته بالشكل:

1234567839

2. تغطية التفرعات (Branch Coverage)

في تغطية التفرعات (Branch Coverage) ، يكون المطلوب هو تغطية جميع التفرعات الموجودة داخل البرنامج. لدينا التفرعات التالية:

- التفرع الأول عند العقدة (3)
- التفرع الثاني عند العقدة (4)
- التفرع الثالث عند العقدة (6)



المسار السابق: 1234567839 كم فرعاً غطّي؟

لقد غطّي معظم التفرعات، ولكن بقي لدينا فرعان لم تتم تغطيتهما، وهما:

حالة (False) الخاصة بالعقدة (6)

حالة (False) الخاصة بالعقدة (4)

لذلك نحتاج إلى مسار آخر 12346839 :

وبالتالي أصبح لدينا مساران فقط استطاعا تحقيق تغطية التفرعات (Branch Coverage)

الآن سنؤجل موضوع تغطية المسارات (Path Coverage) قليلاً.

لكن السؤال هنا:

هل وصلنا فعلاً إلى ما نريده؟

هل انتهت القصة عند هذا الحد؟

ليس تماماً.

ما نحتاجه الآن هو حالات الاختبار (Test Cases)

أي أننا نريد الآن توليد حالات اختبار تجعل البرنامج يسلك هذا المسار الأول تحديداً 1234567839 :

عادةً عندما نريد استخراج حالات الاختبار من اختبار المسارات (Path Testing) ، فإننا نستخدم ما يسمى:

معادلة المسار (Path Equation)

3. معادلة المسار (Path Equation)

وهي عبارة عن معادلة تحل كما لو كنت تحل معادلة من الدرجة الأولى أو معادلة تحتوي عدة مجاهيل، والهدف منها إيجاد القيم التي تحقق شروط المسار.

في النهاية نحصل على قيم والتي تجعل التنفيذ يسير وفق المسار المطلوب.

لكن السؤال هنا:

من الذي يحدد الاتجاه الذي يجب أن نسلكه؟

من الذي يحدد هذه المعادلة أصلاً؟

الجواب هو:

الشروط (Conditions)

فالشرط هو الذي يخبرك:

■ إذا تحقق الشرط تتحرك إلى اليمين

■ وإذا لم يتحقق تتحرك إلى اليسار

بعد ذلك تبدأ بالرجوع عكسياً (Reverse Analysis): تبدأ من الشرط ثم تعود إلى الخلف حتى تصل إلى القيم المطلوبة.

■ عند الشرط الأول لدينا: $0 < y + x$

لا يوجد قبل المتغير (y) أي تعديل على قيمته، لذلك أول نقطة يجب تحقيقها هي:

■ كلا المتغيرين موجبين أو ■ أحدهما موجبًا بدرجة أكبر من الآخر

ولكي نكمل العملية، نستنتج أن الحد الأدنى المطلوب لقيمتي (x, y) هو أن تكونا أكبر من الصفر. ثم نلاحظ نقطة مهمة:

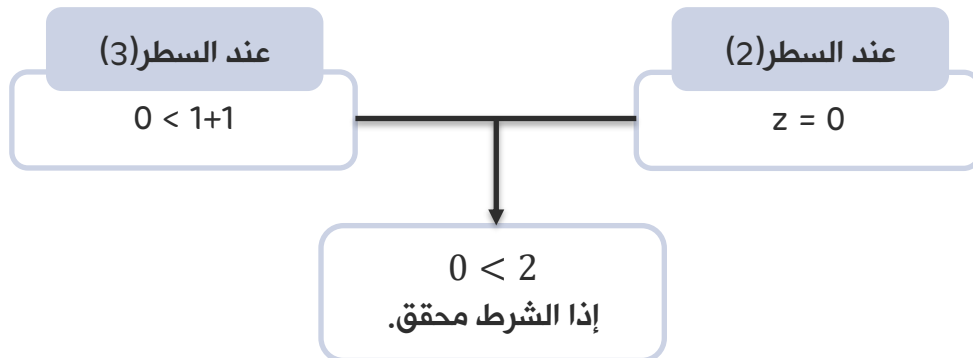
لا توجد أي عملية تعديل مشتركة بين قيمتي (x, y) أي لا يوجد ترابط (Interconnection) يؤدي إلى تعديل إحدى القيمتين بواسطة الأخرى.

لذلك يمكننا اختيار القيم: $x=1$, $y=1$

والآن سنجرب ما الذي سيحدث K هذه الحالة تسمى:

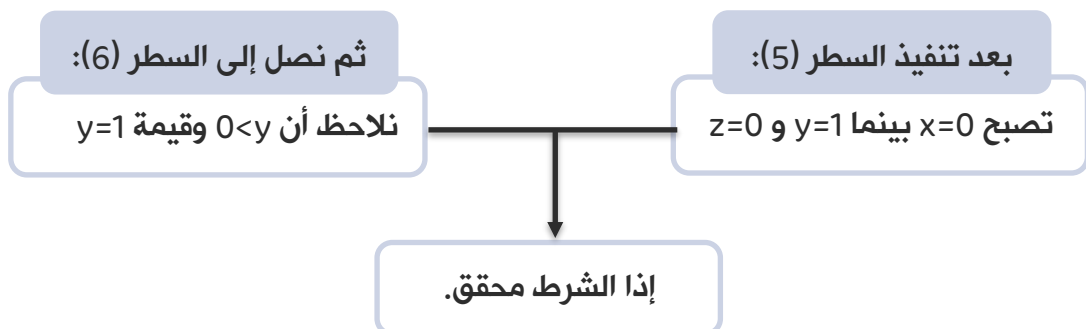
حالة اختبار محفزة للمسار (Trigger the Testing Path Test Case)

أي أنها تجعل التنفيذ يسير ضمن هذا المسار المحدد.

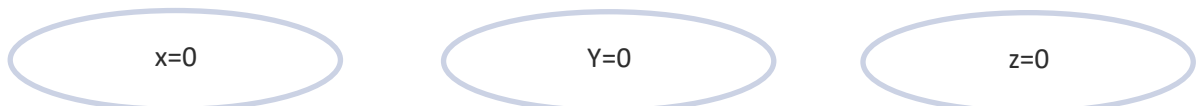


■ ندخل إلى الشرط التالي: $0 < x$

وقيمة x تساوي 1 ($x=1$): إذا الشرط محقق.



بعد تنفيذ العملية في السطر (7) تصبح:



ثم نصل إلى السطر (8) ويتم حساب: $z = x + y + z$

أي: $z = 0 + 0 + 0 = 0$

ثم نعود إلى شرط الحلقة مرة أخرى لكننا ذكرنا سابقاً أننا نريد الدخول إلى الحلقة مرة واحدة فقط.

ولهذا اخترنا: $x=1, y=1$ لأن $x=0, y=0$

وبالتالي:

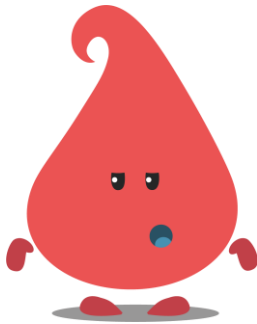
$$0 < x + y$$

لا يتحقق.

لذلك ينتقل التنفيذ إلى السطر (9)، وتكون قيمة: $z=0$ ← إذا حالة الاختبار: (1,1,0)

تؤدي إلى تنفيذ المسار المطلوب.

الآن نريد تطبيق الفكرة نفسها على المسار الثاني 12346839.



المسار الثاني يعتمد على شرطين أساسيين:

- أولاً: يجب أن يتحقق شرط الحلقة
- ثانياً: يجب ألا يتحقق الشرط الأول
- ثالثاً: يجب ألا يتحقق الشرط الثاني

شرط الحلقة هو:

$$0 < x + y$$

وقد ذكرنا سابقاً أن ذلك يحدث عندما:

- تكون قيمة x أكبر من الصفر
- أو
- تكون قيمة y أكبر من الصفر

بشرط أن يكون مجموعهما في النهاية أكبر من الصفر.

لكن هنا تظهر مشكلة، إذا افترضنا:

$$0 < x$$

فسوف يتحقق الشرط الأول، بينما نحن نريد أن يكون غير محقق.

وإذا افترضنا:

$$0 < y$$

فسوف يتحقق الشرط الثاني، بينما نحن لا نريده أن يتحقق أيضاً، بالتالي يصبح لدينا وضع إجباري:

أحد المتغيرين يجب أن يكون أكبر من الصفر لتحقيق شرط الحلقة، لكن هذا سيؤدي تلقائياً إلى تحقيق أحد الشرطين الداخليين.

إذن هذا المسار غير قابل للتنفيذ، وهذا النوع يسمى: **المسار غير الممكن تنفيذه (Infeasible Path)**

هل يؤثر هذا على نسبة التغطية (Coverage Percentage) ؟

الجواب: لا، لأن هذا المسار مستحيل التحقيق أصلاً.
تخيل أنك استخرجت جميع المسارات الممكنة نظرياً، ثم حاولت تغطيتها بحالات اختبار (Test Cases)، لكنك لم تستطع إيجاد حالة اختبار واحدة تحقق هذا المسار.
في هذه الحالة لا يعني ذلك أنك فشلت في الاختبار، وإنما يعني أن هذا المسار غير قابل للتحقيق من الأساس.
ولهذا فإن حساب نسبة تغطية المسارات يكون بالشكل التالي:

نسبة التغطية = مجموع عدد المسارات الممكنة — عدد المسارات غير القابلة للتحقيق

وبالتالي فإن المسارات غير القابلة للتحقيق (infeasible Paths) لا تدخل ضمن حساب نسبة التغطية.
لكن هنا تظهر **مشكلة** أخرى:
يوجد جهد إضافي يجب بذله لاكتشاف هذه المسارات غير القابلة للتحقيق، وهذا الموضوع يُعد من الموضوعات المهمة في أبحاث الاختبار (Testing Research)
الآن، بما أن هذا المسار غير صالح، فلنبحث عن مسارات أخرى تحقق تغطية التفرعات (Branch Coverage)
تصبح المسارات المطلوبة:

1. المسار الأول: 123456839

2. المسار الثاني: 123467839

وبذلك نستطيع تحقيق 100% تغطية للتفرعات (Branch Coverage)
والآن أصبح المطلوب هو تغطية المسارات (Path Coverage)
إذا تجاهلنا المسار 1239، فكم احتمال انتقال يوجد نحو العقدة (8)؟
الآن إذا أردنا التفكير بالحد الأدنى من المسارات، فمن النظرة الأولى نستطيع ملاحظة وجود خمسة مسارات على الأقل.
لكن المشكلة ليست هنا، المشكلة أن هذه مجرد رؤية أولية، لأن تعريف المسار الحقيقي هو:

احتمال الانتقال الكامل من نقطة الإدخال (Input) حتى نقطة الإخراج (Output)

وطالما توجد لدينا حلقة (Loop)، وشرط الحلقة هو:

$$0 < y + x$$

فإن تحقق هذا الشرط يعتمد على قيم x , y وبالتالي توجد احتمالات كثيرة جداً للعودة إلى الحلقة والدخول إليها مرة أخرى.

تخيلوا السيناريو التالي:

قلنا نظرياً إن لدينا داخل الحلقة أربع مسارات أو احتمالات انتقال.
إذا دخلنا دورة ثانية داخل الحلقة فسيظهر لدينا أربع احتمالات جديدة.

وإذا دخلنا عشر دورات، فسنحصل على عدد كبير جداً من المسارات.
وهنا تظهر مشكلة تُعرف باسم:

انفجار المسارات (Path Explosion)

انفجار المسارات (Path Explosion)

أي أن عدد المسارات يزداد بشكل هائل جداً، ولهذا السبب لا نستطيع عملياً اختبار جميع المسارات الممكنة،
ولحل هذه المشكلة نلجأ إلى مفهوم آخر يسمى:

اختبار المسارات الأساسية (Basic Path Testing)

القاعدة بسيطة:

في هذا المثال يمكن ملاحظة أن عدد المسارات الأساسية يساوي عدد الشروط + 1.

الحل الأول: في مثالنا لدينا:

عدد الشروط = 3 ← إذا: عدد المسارات الأساسية = $3 + 1 = 4$ مسارات

هذه إحدى الطرق المستخدمة لتقليل انفجار عدد المسارات، وفعلياً لدينا المعيار التالي:

الحل الثاني للمشكلة يعتمد على قواعد اختبار الحلقات (Loop Testing Rules) ، فبدل أن نحاول اختبار جميع احتمالات الحلقة، نقوم باختبارها وفق معايير محددة.

■ Cyclomatic Complexity :

يُستخدم لحساب عدد المسارات المستقلة (Independent Paths) داخل البرنامج، حيث يوجد أكثر من طريقة لحسابه، وأشهر معادلتين هما:

1. الطريقة الأولى : $V(G) = P + 1$

حيث:

■ $V(G) = \text{Cyclomatic Complexity}$

■ $P = \text{عدد العقد الشرطية}$

2. الطريقة الثانية : $V(G) = E - N + 2$

تعتمد مباشرة على مخطط تدفق التحكم (CFG)

حيث:

■ $E = \text{عدد الحواف (Edges)}$

■ $N = \text{عدد العقد (Nodes)}$

ويجب أن تعطي الطريقتان النتيجة نفسها عند تطبيقهما على المخطط نفسه.
في مثالنا الحالي لدينا ثلاث عقد شرطية:



وبالتالي $V(G) = 3 + 1 = 4$

أي أن عدد المسارات المستقلة (Independent Paths) المطلوبة لاختبار المسارات الأساسية يساوي أربعة مسارات.

لماذا نستخدم المعادلتين؟

عندما يكون الكود آمناً مباشرة ويسهل عدّ الشروط، تكون معادلة:

$V(G) = P + 1$ هي الأسرع.

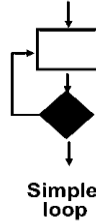
أما عندما يكون لدينا مخطط CFG مرسوم، فيمكن استخدام:

$V(G) = E - N + 2$ للتحقق من صحة الحساب أو في الحالات التي يصعب فيها عدّ العقد الشرطية بشكل مباشر.

ملاحظة مهمة: لا يعني الحصول على قيمة Cyclomatic Complexity تساوي 4 أننا سنختبر جميع المسارات الممكنة داخل البرنامج، خاصة عند وجود حلقات (Loops)، لأن عدد المسارات الفعلي قد يكون كبيراً جداً أو حتى غير منتهٍ بسبب ظاهرة **Path Explosion** وإنما تعني هذه القيمة أننا نحتاج إلى أربعة مسارات مستقلة (Independent Paths) على الأقل لتحقيق Basic Pat Testing.

أنواع الحلقات

1. الحلقة البسيطة (Simple Loop)



طالما يوجد شرط داخل الحلقة، فإن هذا الشرط يولد مباشرة مجالاً (Range)

مثلاً إذا كان مجال الحلقة: [3 , 8]

فإذا كان المجال صغيراً ولدينا وقت كافٍ، يمكن اختبار جميع القيم.

أما إذا لم يكن لدينا وقت، فإننا نختار نقطة واحدة فقط للدخول إلى الحلقة.

لكن أي نقطة نختار؟

■ لا نختار أول قيمة ولا آخر قيمة لأن الأخطاء غالباً تظهر عند الحدود.

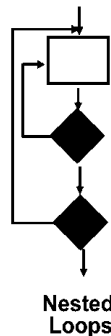
فمثلاً نسيان علامة المساواة (=) قد يحول المجال من مغلق إلى مفتوح أو العكس.

ولهذا نختار غالباً منتصف المجال.

في المثال [3 , 8] يمكن اختيار 5 ، ثم إذا توفر وقت إضافي نختبر أيضاً القيم الموجودة قرب الحدود مثل:

2 و 9 لأن القاعدة المهمة تقول: معظم الأخطاء تظهر عند حدود المجال.

2. الحلقات المتداخلة (Nested Loops)



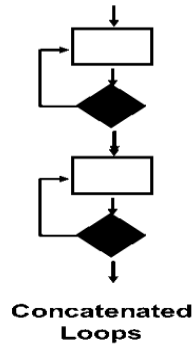
هذه أكثر تعقيداً، وتطبق عليها القواعد نفسها ولكن بالتدريج، حيث لدينا الحلقة الكبيرة والحلقة الصغيرة داخلها.

فنختار أولاً قيمة تدخلنا إلى الحلقة الصغيرة بصورة مؤكدة.

بعد ذلك نختبر الحلقة الصغيرة كما لو كانت حلقة بسيطة (Simple Loop)

ثم بعد الانتهاء منها نعود لاختبار الحلقة الكبيرة بالطريقة نفسها.

3. الحلقات المتسلسلة (Concatenated Loops)



وهي حلقات تأتي واحدة بعد الأخرى:
حلقة ← ثم حلقة أخرى ← ثم حلقة أخرى.
ويتم التعامل معها كل حلقة على حدة

مثال 2:

لدينا هذا المسار الذي لا يحتوي على حلقات، فالمطلوب هو استخراج معايير التغطية (Coverage Criteria) للوصول إلى حالات الاختبار (Test Cases).

```
1. input(x,y);
2. x=x+1;
3. if(x>0)
4.   x=x+1;
   else
5.   x=x-1;
6. if(y>0)
7.   y=y+1;
   else
8.   y=y-1;
9. output(x,y);
```



"في البداية سترى الكود كسطور

وبعد فترة ستبدأ ترى السلوك والمنطق خلفه"

1. أولاً: بناء الـ CFG

لدينا شرطان:

1. الشرط الأول: $\text{if}(x > 0)$

ينتج:

العقدة 4 ← True

العقدة 5 ← False

2. الشرط الثاني: $\text{if}(y > 0)$

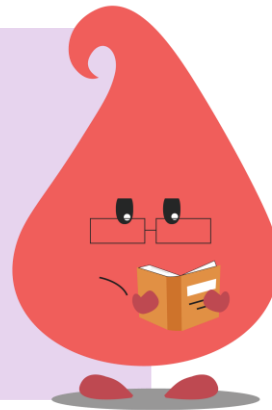
ينتج:

العقدة 7 ← True

العقدة 8 ← False

الـ CFG النهائي:

الفرق بين من ينجح ومن يتطور فعلاً
أن الثاني يصبر على مرحلة عدم الفهم.



2. ثانياً: استخراج جميع المسارات (Paths)

لأن لدينا شرطين مستقلين، الشرط الأول له احتمالان، والشرط الثاني له احتمالان، إذاً: $2 \times 2 = 4$ Paths وتكون المسارات الكاملة كالتالي:

Path 2

الأول True

والثاني False

1-2-3-4-6-8-9

Path 1

الشرطان True

1-2-3-4-6-7-9

Path 4

الشرطان False

1-2-3-5-6-8-9

Path 3

الأول False

والثاني True

1-2-3-5-6-7-9

3. ثالثاً: Statement Coverage :

ما المطلوب؟ تنفيذ جميع التعليمات مرة واحدة على الأقل.

هل مسار واحد يكفي؟

لنجرّب المسار (1-2-3-4-6-7-9):

نلاحظ أن هذا المسار لم ينفذ 5 و 8، إذا ليس 100%

أقل عدد مسارات لتحقيق Statement Coverage نحتاج :

مسار يمر عبر 4 و مسار يمر عبر 5 وكذلك مسار يمر عبر 7 و مسار يمر عبر 8

اختيار ذكي

كل مهندس قوي مرّ بمرحلة

كان يشعر فيها أن الـ Paths لا تنتهي



نأخذ:

P1 = 1-2-3-4-6-7-9

P4 = 1-2-3-5-6-8-9

الآن جميع العقد تم تنفيذها.

4. رابعاً: Branch Coverage :

لدينا شرطان:

1. الشرط الأول: $\text{if}(x>0)$

الفروع:

True ← العقدة 4

False ← العقدة 5

2. الشرط الثاني: $\text{if}(y>0)$

الفروع:

True ← العقدة 7

False ← العقدة 8

عدد الفروع الكلي 4 Branches، هل P1 وحده يكفي؟

ينفذ:

4 ← 3

7 ← 6

إذن: 50%

لكن لا ينفذ:

5 ← 3

8 ← 6

ولتحقيق 100% Branch Coverage، نستخدم: P1 + P4 لأنهما يغطيان جميع الفروع.

5. خامساً: Path Coverage :

عدد المسارات الكلي 4 Paths =

لتحقيق 100% Path Coverage

يجب تنفيذ:

كلها

P4

P3

P2

P1

6. سادساً: استخراج Test Cases :

الآن نريد قيم تجعل التنفيذ يسلك كل مسار.

لكن انتبه قبل الشرط الأول يوجد: $x = x + 1$

إذا الشرط الحقيقي: $x + 1 > 0$ ، أي $x > -1$

وأيضاً:

الشرط الثاني يعتمد على y ، لا يوجد تعديل قبل الشرط، إذا يبقى $y > 0$.

P1 J Test Case ■

المطلوب:

$$y > 0$$

$$x > -1$$

مثال: $X=0$, $Y=1$

P2 JTest Case ■

المطلوب:

$$y \leq 0$$

$$x > -1$$

مثال: $X=0$, $Y=0$

P3 JTest Case ■

المطلوب:

$$y > 0$$

$$x \leq -1$$

مثال: $X=1$, $Y=1$

P4 JTest Case ■

المطلوب:

$$y \leq 0$$

$$x \leq -1$$

مثال: $X=-1$, $Y=0$

فيكون الجدول النهائي:

Path	Execution Path	Condition
P1	1-2-3-4-6-7-9	$x > -1$, $y > 0$
P2	1-2-3-4-6-8-9	$x > -1$, $y \leq 0$
P3	1-2-3-5-6-7-9	$x \leq -1$, $y > 0$
P4	1-2-3-5-6-8-9	$x \leq -1$, $y \leq 0$

ثانياً: تدفق البيانات (Data Flow)

الفكرة هنا أنه طالما الذي يحدد طبيعة الانتقال بين المسارات المختلفة داخل البرنامج هو قيم المتغيرات (Variables)، فمن الممكن تتبع هذه المتغيرات واكتشاف الأخطاء. أي تتبع:

أين تم تعديلها؟

أين تمت قراءتها؟

أين تمت كتابة القيمة؟

فائدة: استخدام (Use): نقصد عملية قراءة المتغير.

تعريف (Definition): إعطاء المتغير قيمة جديدة أو تعديل قيمته.

❧ في عملية القراءة يوجد نوعان:

■ استخدام حسابي (Computational Use - C-use)

وهو استخدام المتغير داخل معادلة.

مثل: $x = x + 1$ هنا يتم استخدام قيمة x داخل معادلة حسابية.

■ استخدام شرطي (Control Use - P-use)

وهو قراءة قيمة المتغير من أجل تقييم شرط.

مثل: $x < 1$ هنا تتم قراءة قيمة x لاتخاذ قرار هل نذهب يميناً أم يساراً؟

الفكرة الأساسية هي علينا دائماً تتبع العلاقة بين التعريف (Definition) والاستخدام (Use) أي أن كل تعريف جديد لمتغير يجب أن نعرف أين تم استخدامه لاحقاً وبمجرد إنشاء تعريف جديد للمتغير، فإن سلسلة الاستخدام السابقة تنتهي، وتبدأ سلسلة جديدة.

الآن لنطبق الفكرة على المثال التالي:

```
1. input(x,y);
2. x=x+1;
3. y=y+1;
4. if(x>0)
5.   y=y+1;
   else
6.   y=y+2;
7. output(x,y);
```



■ بالنسبة للمتغير x لدينا:

أول تسلسل "First definition"

- أول تعريف (Definition) : أول مرة تم تعريف X عند العقدة 1
- أول استخدام (Use) : وأول مرة تم استخدامها عند العقدة 2 وذلك لأنه تمت قراءتها عند هذه العقدة

أما التسلسل الثاني "Second definition"

تم جمع المتغير عند السطر الثاني في عملية كتابة، وبعدها ذهب للعقدة 4 لعملية القراءة، وهنا لدي قرارين: في حال تحقق الشرط، تذهب ل5، وفي حال عدم تحقق الشرط تذهب للعقدة 6، ومن ثم لدي اخراج ال 7. التسلسلات أصبحت: 2->7 , 2->4->6 , 2->4->5 , 1->2 الأساسية.

■ بالنسبة للمتغير y لدينا التسلسل:

- هو 1->3 وبعدها 6->7 , 5->7 , 3->6 , 3->5
- 5->7 وذلك لأن لدي definition في ال 5 وبعدها قراءة في ال 7.
- 6->7 أي definition في ال 6 وبعدها قراءة في ال 7.

اختبار الصندوق الأسود (Black Box Testing)

هو اختبار يعتمد على الميزات والوظائف (Feature-Based Testing) أو اختبار من البداية إلى النهاية (End-to-End Testing)

نفترض أن لدينا تطبيقاً جاهزاً، يحتوي على واجهات، يحتوي على قاعدة بيانات، يحتوي على وظائف مختلفة عندها تبدأ باستخدام الواجهات وتجربة التطبيق والتأكد من صحة النتائج وهنا نقوم بعملية اختبار (Testing)

لكن السؤال الحقيقي: هل أنت تعرف بدقة ماذا تدخل إلى النظام وماذا يخرج منه؟

في كثير من الحالات يكون الاختبار أقرب إلى اختبار عشوائي (Random Testing) لذلك حتى في اختبار الصندوق الأسود نحتاج إلى اختيار حالات اختبار (Test Cases) بطريقة مدروسة.

الفكرة هنا مختلفة قليلاً، بدل أن ننظر إلى التعليمات البرمجية (Code Statements) ، يمكننا اعتبار كل واجهة أو شاشة داخل التطبيق على أنها عقدة (Node) والانتقال من واجهة إلى أخرى يصبح انتقالاً من عقدة إلى عقدة. لكن هنا يوجد اختلاف مهم:

في مخطط تدفق التعليمات البرمجية كانت الانتقالات تعتمد على شروط منطقية (Conditions)



أما هنا فعملية الانتقال قد تعتمد على:

- ضغط زر
- اختيار قائمة
- إدخال بيانات
- تفاعل المستخدم مع التطبيق

وبالتالي يمكننا رسم مخطط تدفق الميزات والانتقالات (Feature Flow Graph) ، ومن هنا تظهر مفاهيم مشابهة لما رأيناه سابقاً، لدينا:

1. تغطية الحالات (State Coverage)

وهي من أخطر المفاهيم وأهمها، لماذا؟
لأن الحالة (State) هنا لا تعني مجرد موقع داخل الكود، وإنما:
حالة التطبيق بالكامل بناءً على قيم متغيراته الأساسية (main variable value)

2. تغطية الانتقالات (Transition Coverage)

أي تغطية جميع الانتقالات بين العقد.

3. تغطية المسارات أو تتبع التنفيذ (Path Coverage / Trace Coverage)

وهي تتبع الانتقالات اللازمة للوصول إلى الناتج النهائي المطلوب.

مثال:

إذا أردت تنفيذ عملية طلب (Order) ، فما هي احتمالات الانتقال الممكنة؟

قد توجد:



لكن كل مرحلة قد تحتوي احتمالات متعددة للانتقال هنا لا تصبح القضية مجرد انتقال من نقطة إلى أخرى، بل تدخل عوامل أخرى مثل:

- حالة التطبيق الحالية
- مجال قيم المتغيرات
- تفاعل المستخدم مع العناصر المختلفة

ولهذا، عندما نريد اختيار المدخلات (Inputs) والمخرجات (Outputs) بطريقة صحيحة لاستخراج حالات الاختبار (Test Cases)، فإننا نعتمد على مجموعة من القواعد:

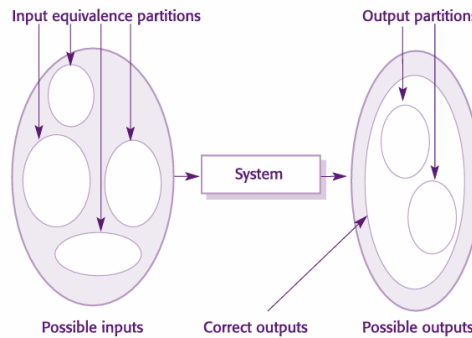
القاعدة الأولى:

حتى التطبيق نفسه يمكن تمثيله على شكل مخطط (Graph) ، ثم تطبيق معايير التغطية (Coverage Criteria) بالطريقة نفسها التي استخدمناها في اختبار الصندوق الأبيض (White Box Testing) ، ولكن بصورة مختلفة تناسب مع مستوى التطبيق الكامل.

القاعدة الثانية: اختيار المدخلات المناسبة لمسار محدد

المشكلة هنا أننا لا نملك قيمة إدخال واحدة فقط، بل لدينا مجموعة كبيرة من المدخلات الممكنة (Input Range) وكل مسار قد يحتاج مجموعة مختلفة من القيم. كيف أختار المدخلات المناسبة و هل سأجرب جميع الاحتمالات؟ في الواقع، هذا غير ممكن عملياً.

القاعدة الثالثة: التقسيم المتكافئ (Equivalent Partitioning)



■ التقسيم المتكافئ (Equivalent Partitioning)

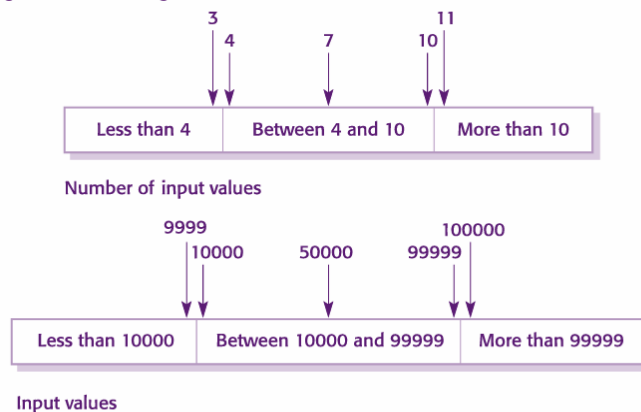
وهو أنه كل مجموعة من المدخلات التي تؤدي إلى النتيجة نفسها تعامل على أنها مجموعة واحدة.

ملاحظة: بدلاً من التعامل مع عدد غير متناهٍ من المدخلات، سأعامل مع عدد محدود من مجموعات الإدخال،

فكل مجموعة تحتوي على عدد كبير جداً من القيم التي تقودني إلى النتيجة نفسها.

فأول خطوة نقوم بها هي التقسيم المتكافئ (Equivalent Partitioning) ، بعد ذلك تأتي الخطوة الثانية كيف سأختبر كل مجموعة؟

■ تحليل القيم الحدية (Boundary Value Analysis)



ما يميز كل مجموعة عن الأخرى هو حدودها فالحدود هي التي تفصل مجموعة عن أخرى.
وحتى الآن ما زلنا نتحدث عن أمثلة رقمية، لكن لو كانت البيانات:

أسماء مستخدمين

نصوصاً

أنواع ملفات

كلمات مرور

فالفكرة نفسها تطبق.

كل ما تحدثنا عنه حتى الآن ما يزال ضمن اختبار الصندوق الأسود (Black Box Testing)
لكن هناك أنواع أخرى مثل:

- اختبار الأداء (Performance Testing)
- اختبار الأمان (Security Testing)
- اختبار الحمل (Load Testing)
- اختبار الإجهاد (Stress Testing)
- اختبار الاستعادة (Recovery Testing)
- اختبار القائم على النموذج (Model-Based Testing): يستخدم نموذجاً سلوكياً لتحديد المدخلات والمخرجات المتوقعة وتتبع انتقال البرنامج بين الحالات.

انتهت المحاضرة

